



Low-Latency Silkworm Disease Detection

on FPGA using Classical Machine Learning

MYZ307E · Machine Learning for Electrical & Electronics Engineering

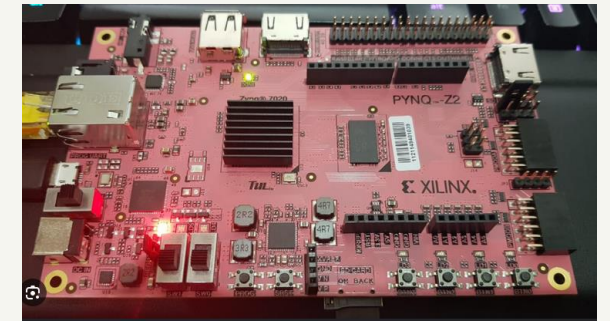
Talha Sarıkaya · Term Project, May 2026



Problem & Motivation

Why automated, edge-side disease screening on the rearing shelf?

- One missed worm = one lost tray



Why this matters

\$ losses

Whole-tray

from one infected worm

Latency budget

< 1 ms

per worm, conveyor pace

Device

PYNQ-Z2

Xilinx Zynq-7020, no GPU

Reference Paper & Structural Baseline

Anchoring our contribution against an existing classical-ML pipeline

Academic reference

- Binson V A & Manju G, “Automated Disease Detection in Silkworms Using Machine Learning Techniques,” ASSET, vol. 6(4), 2024 [1].

Structural code baseline

- github.com/haxybaxy/PlantDiseaseDetection — classical HOG / LBP + SVM / RF pipeline for agricultural imagery; we re-used its structure and adapted it to silkworms.



Our goal: match the paper's accuracy AND high disease recall in a feature pipeline that synthesises onto a PYNQ-Z2 streaming dataflow.

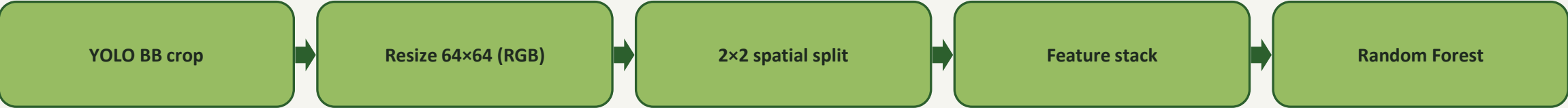
Dataset & Preprocessing

Strict, leakage-free pipeline from Roboflow YOLO export to RF input

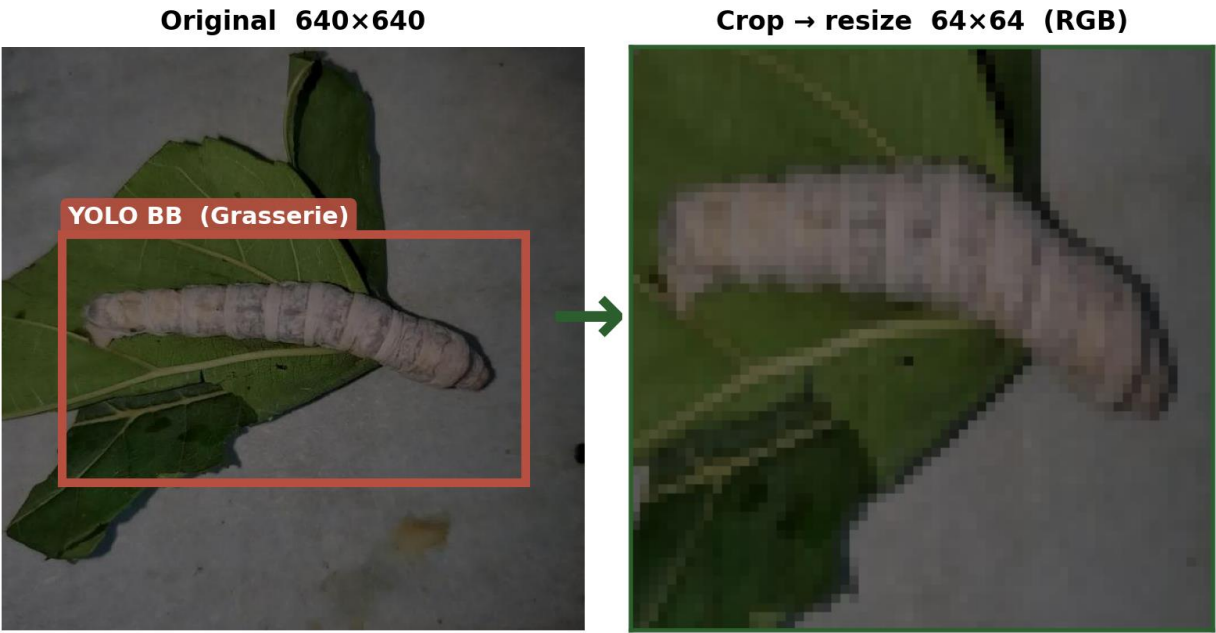
Split sizes after bounding-box crop

Split	Grasserie (0)	Healthy (1)	Total
Train	4 000	4 174	8 174
Valid	264	249	513
Test	249	271	520

Preprocessing pipeline (kept RGB this iteration)



Step 4 — bounding-box crop removes leaf / shelf background



Improvement #1 — Recall-Optimised Training

Make the classifier afraid of missing disease, not afraid of false alarms

- Bias the loss + lower the threshold. No FPGA cost

2.5×

class_weight on Grasserie

- Applied to a minimal LBP-only baseline this gives 98.8 percent recall but only 70.7 percent precision

0.40

P(Grasserie) decision threshold

0 LUT

added on the FPGA side

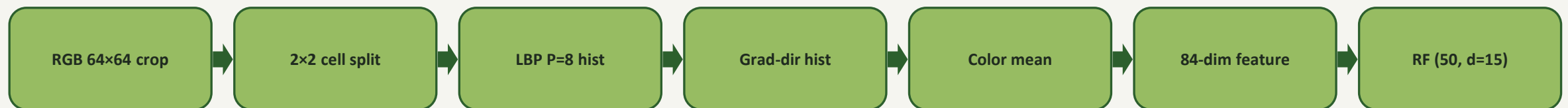
Improvement #2 — Feature Co-Design for FPGA

Three integer feature families on a shared 3-row line buffer

2×2 spatial pooling — 4 cells × (10 LBP + 8 grad + 3 color) = 84 features



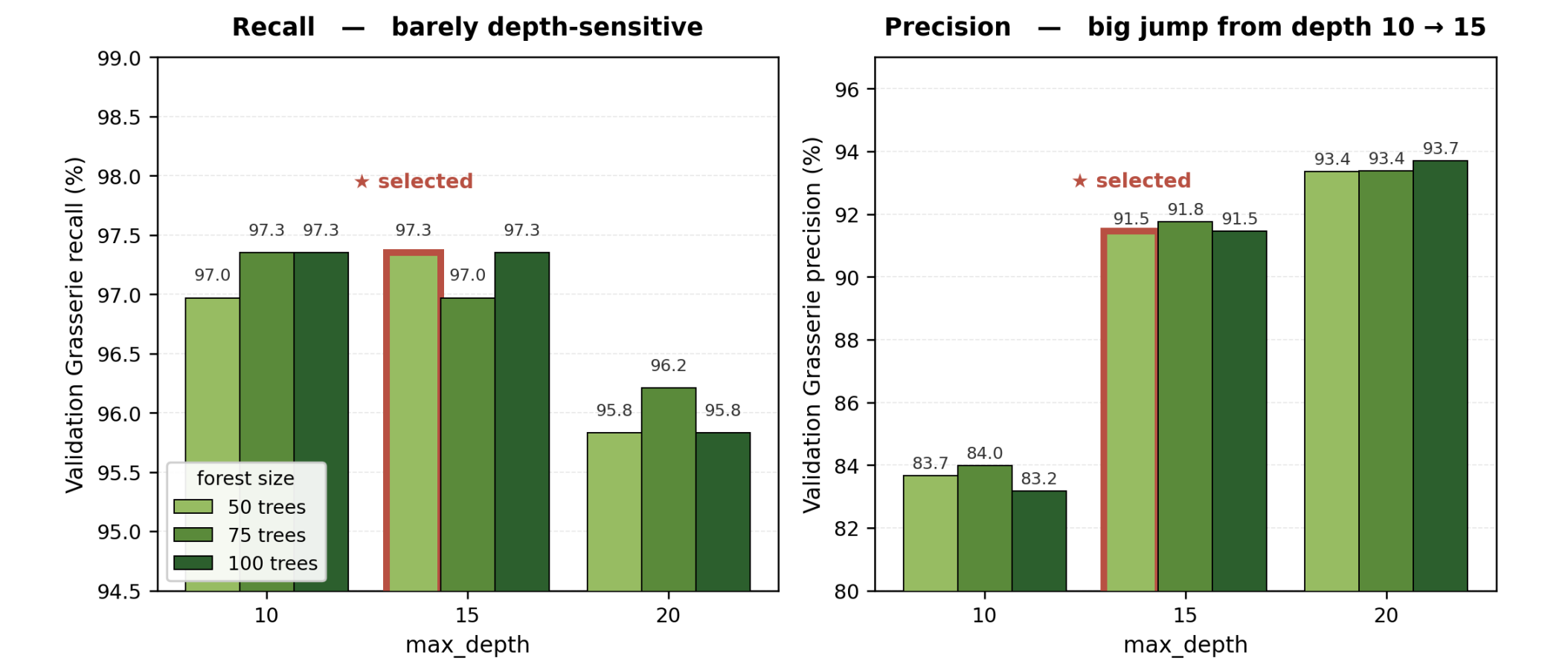
*+ 2×2 spatial pooling + gradient histogram + per-cell color
= 84 integer features. Estimated: 18k LUTs.*



Hyperparameter Selection (Validation Set)

3 tree counts × 3 depths = 9 configs; ranked by validation Grasserie recall

■ Rule: max recall → max precision → smallest forest. Winner: 50 trees, depth 15.



Selection rule: max recall → max precision → smallest forest. Tied recall at 97.35 % across four configs; (50 trees, d=15) wins step 2 and step 3 simultaneously.

Results — Reference vs Ours (Held-out Test Set)

Feature co-design lifts every metric while keeping the FPGA budget intact

Method	Features	Acc %	Recall G %	Prec G %	F1 G %	Inference / dataset
Reference [1] — HOG + KPCA + SVM	many	93.16	93.38	91.94	92.65	CPU only · 1 242 imgs (different set)
Ours v1 — LBP-only + RF(50, d=10)	10	79.81	98.80	70.69	82.41	0.053 ms · Roboflow v1 (8 975)
Ours v2 — LBP + grad + color, 2×2 cells + RF(50, d=15)	84	91.54	95.58	87.82	91.54	0.027 ms · Roboflow v1 (8 975)

	Pred: Gr.	Pred: Hl.
Grasserie (249)	246	3
Healthy (271)	102	169

v1 (LBP-only): high recall, low precision

	Pred: Gr.	Pred: Hl.
Grasserie (249)	238	11
Healthy (271)	33	238

v2 (full features): both high

Analysis & Discussion

v2 is a Pareto improvement over v1 — and very close to the paper's accuracy

+11.7 pp accuracy

+17.1 pp precision

−3.2 pp recall

½ inference time

Conclusions & Future Work

What this term project closed, and what it sets up for the senior project

What this term project delivered

- Two-step engineering arc: recall-optimised training (v1), then feature co-design (v2).
- v2 test result: 91.54 % accuracy, 95.58 % Grasserie recall, 87.82 % precision; only 1.6 pp below the reference paper's overall accuracy.
- All-integer feature stack (LBP + grad-hist + color), 2x2 pooling, ~18k LUT FPGA estimate — well under the Zynq-7020 budget.

Future work (graduation project)

- Hand-write random_forest.v in Verilog RTL: each tree = combinational nested mux; majority-vote via adder tree. Architecture is designed by hand.
- Hand-write LBP, Sobel-gradient and per-cell color-mean modules as AXI-Stream IP cores on top of a 3-row line buffer.
- Synthesise on PYNQ-Z2 (Zynq-7020) with Vivado; target physical latency $\leq 100 \mu\text{s/crop}$.

GitHub: <https://github.com/yoctocandela/silkworm-fpga-ml> Dataset: Roboflow Universe / Silkworm Annotation · Silkworm Diseases v1